Helm — 3–4 YOE Scenario-Based Q&A

1. Helm chart install is failing. What will you check?
First, I check the Helm error message to understand where the installation failed.
Then I validate the chart, values, Kubernetes connection, namespace, and required resources.
I also check whether the chart has template or dependency issues.
Based on the error, I fix the configuration and run the installation again.
Simple ga gurthupettuko:
Helm install fail → error chudu → chart/values check cheyyi → Kubernetes/dependency verify cheyyi → fix and install again.
Eng: Helm install fails → check the error → verify chart and values → check Kubernetes/dependencies → fix and install again.
Example:
helm install myapp ./mychart -n dev → Helm install chesthu exact error enti ani chudu.
helm lint ./mychart → Chart lo syntax or configuration mistakes unnaya check cheyyi.
helm template myapp ./mychart -f values.yaml → Kubernetes ki generate ayye YAML correct ga undho check cheyyi.
kubectl get pods -n dev → Install tarvatha Pods create ayyaya ani verify cheyyi.
Issue fix chesi → helm install myapp ./mychart -n dev → Chart successfully install ayyindo verify cheyyi.

---

2. Helm upgrade is failing. What will you check?
First, I check the Helm upgrade error and identify which resource is causing the failure.
Then I compare the current values with the previous working release.
I also check Kubernetes events, templates, image, configuration, and resource changes.
After identifying the issue, I fix it and retry the upgrade or rollback if required.
Simple ga gurthupettuko:
Upgrade fail → error chudu → old vs new values compare cheyyi → Kubernetes/resource changes check cheyyi → fix or rollback.
Eng: Helm upgrade fails → check the error → compare old and new values → verify resource changes → fix or rollback.
Example:
helm upgrade myapp ./mychart -n dev -f values.yaml → Upgrade enduku fail ayyindo error chudu.
helm history myapp -n dev → Previous working release revision enti ani check cheyyi.
helm get values myapp -n dev → Current release lo ye values use ayyayo verify cheyyi.
kubectl get events -n dev --sort-by=.lastTimestamp → Kubernetes lo upgrade failure ki related events chudu.
Issue fix ayyaka → helm upgrade myapp ./mychart -n dev -f values.yaml → Upgrade successful ga ayyindo verify cheyyi.

---

3. Helm upgrade succeeded but application is not working. What will you check?
First, I verify the Helm release status and the resources created by the release.
Then I check Pod status, logs, events, Services, configuration, and the deployed image version.
I also compare the current release with the previous working release.
Based on the issue, I fix the configuration or roll back to the last known-good release.
Simple ga gurthupettuko:
Upgrade success but app fail → release status chudu → Pods/logs/Service check cheyyi → old release tho compare cheyyi → fix or rollback.
Eng: Upgrade succeeds but app fails → check release status → verify Pods, logs, and Service → compare the previous release → fix or rollback.
Example:
helm status myapp -n dev → Helm release deployed status lo undho check cheyyi.
kubectl get pods -n dev → Application Pods Running/Ready lo unnaya verify cheyyi.
kubectl logs <pod_name> -n dev → Application enduku work avvatledo error logs chudu.
kubectl get svc -n dev → Service correct ga create ayyinda and ports check cheyyi.
helm history myapp -n dev → Previous working revision identify cheyyi.
Issue fix avvakapothe → helm rollback myapp <revision> -n dev → Last known-good release ki rollback chesi application verify cheyyi.

---

4. Helm works in DEV but fails in PROD. What will you check?
First, I compare the Helm values and configuration used in DEV and PROD.
Then I check namespaces, Secrets, ConfigMaps, image tags, resources, permissions, and cluster differences.
I also verify whether the same chart version and application image are being used.
After finding the environment-specific difference, I fix it and redeploy.
Simple ga gurthupettuko:
DEV work PROD fail → values compare cheyyi → config/secrets/image/resources check cheyyi → environment difference fix cheyyi.
Eng: DEV works but PROD fails → compare values → check config, secrets, image, and resources → fix the environment difference.
Example:
helm get values myapp -n dev → DEV release lo use aina values check cheyyi.
helm get values myapp -n prod → PROD release values tho compare cheyyi.
kubectl get secret -n prod → Required Secrets PROD lo available unnaya check cheyyi.
kubectl get configmap -n prod → PROD ConfigMaps correct ga unnaya verify cheyyi.
helm list -n prod → PROD lo correct chart/release version deploy ayyindo check cheyyi.
Difference fix chesi → helm upgrade myapp ./mychart -n prod -f values-prod.yaml → PROD deployment verify cheyyi.

5. Helm release is stuck in pending state. What will you check?
First, I check the Helm release status and history to understand the previous operation.
Then I check Kubernetes resources, events, and any Helm hooks related to the release.
I also verify whether another Helm operation or deployment is already running.
Based on the cause, I complete, recover, or safely roll back the release.
Simple ga gurthupettuko:
Release pending → status/history chudu → Kubernetes events/hooks check cheyyi → another operation unda chudu → recover or rollback.
Eng: Release pending → check status/history → verify events and hooks → check running operations → recover or roll back.
Example:
helm status myapp -n dev → Release current state pending lo undho check cheyyi.
helm history myapp -n dev → Previous revisions and failed/pending operation details chudu.
kubectl get events -n dev --sort-by=.lastTimestamp → Pending ki Kubernetes events reason enti ani check cheyyi.
kubectl get pods -n dev → Release related Pods stuck ayyi unnaya verify cheyyi.
Issue recover cheyyadam possible kakapothe → helm rollback myapp <revision> -n dev → Known-good revision ki safely rollback cheyyi.

---

6. How do you rollback a failed Helm deployment?
First, I check the Helm release history and identify the last known-good revision.
Then I perform a controlled Helm rollback to that revision.
After rollback, I verify Pods, Services, application health, and release status.
I also monitor the application to confirm that the Production issue is resolved.
Simple ga gurthupettuko:
Helm rollback → history chudu → last good revision identify cheyyi → rollback cheyyi → app health verify cheyyi.
Eng: Helm rollback → check release history → identify the last good revision → roll back → verify application health.
Example:
helm history myapp -n prod → Release history chusi last successful revision identify cheyyi.
helm rollback myapp 4 -n prod → Release ni revision 4 ki rollback cheyyi.
helm status myapp -n prod → Rollback tarvatha release status verify cheyyi.
kubectl get pods -n prod → New Pods healthy ga Running/Ready lo unnaya check cheyyi.
kubectl logs <pod_name> -n prod → Application errors unnaya verify cheyyi.

---

7. Helm values are not being applied. What will you check?
First, I check whether the correct values.yaml file is being used.
Then I verify the value names and YAML structure and check whether another values file or --set is overriding them.
I also render the chart to see the actual values passed to Kubernetes.
Based on the output, I correct the values or override configuration.
Simple ga gurthupettuko:
Values apply avvakapothe → correct values file check cheyyi → key/structure check cheyyi → override unda chudu → rendered output verify cheyyi.
Eng: Values not applied → check the values file → verify keys and structure → check overrides → verify the rendered output.
Example:
helm upgrade myapp ./mychart -f values.yaml -n dev → Correct values file command lo use chestunnama check cheyyi.
cat values.yaml → Values keys and YAML structure correct ga unnaya chudu.
helm template myapp ./mychart -f values.yaml → Final generated YAML lo expected value apply ayyinda verify cheyyi.
helm get values myapp -n dev → Running release actually ye values use chestundo check cheyyi.
Override issue fix chesi → helm upgrade myapp ./mychart -f values.yaml -n dev → New values apply ayyayo verify cheyyi.

---

8. Wrong value is getting picked from values.yaml. What will you check?
First, I check all the values files being passed during the Helm command.
Then I check --set parameters because they can override values from files.
I also verify the exact key path used in the Helm template.
After identifying the override or key mismatch, I correct it and redeploy.
Simple ga gurthupettuko:
Wrong value → values files check → --set check → template key check → mismatch fix cheyyi.
Eng: Wrong value → check values files → verify --set overrides → check the template key → fix the mismatch.
Example:
helm upgrade myapp ./mychart -f values.yaml -f values-prod.yaml -n prod → Multiple values files lo ye file values override chestundo identify cheyyi.
helm upgrade myapp ./mychart --set image.tag=v2 -n prod → --set expected value ni override chestundo check cheyyi.
grep -R "image.tag" ./mychart/templates → Template lo exact value key/path ela use chestunnaro check cheyyi.
helm template myapp ./mychart -f values.yaml --set image.tag=v2 → Final rendered value correct ga undho verify cheyyi.
Mismatch fix chesi → helm upgrade myapp ./mychart -n prod → Correct value deploy ayyindo verify cheyyi.

---

9. How do you manage DEV, QA, UAT, and PROD using Helm?
I maintain a common Helm chart and keep environment-specific values separately.
For example, DEV, QA, UAT, and PROD can have different values files for replicas, resources, endpoints, and configuration.
The chart structure remains reusable across environments.
This reduces duplicate deployment code and keeps environment configuration controlled.
Simple ga gurthupettuko:
One chart → separate environment values → same deployment logic → required environment ki correct values use cheyyi.

Eng: One chart → separate environment values → reuse the same deployment logic → use the correct values for each environment.
Example:
helm upgrade myapp ./mychart -f values-dev.yaml -n dev → DEV environment ki DEV-specific values tho deploy cheyyi.
helm upgrade myapp ./mychart -f values-qa.yaml -n qa → QA ki separate values use cheyyi.
helm upgrade myapp ./mychart -f values-uat.yaml -n uat → UAT configuration tho deploy cheyyi.
helm upgrade myapp ./mychart -f values-prod.yaml -n prod → PROD ki controlled PROD values use cheyyi.
helm list -A → All environments lo Helm releases status verify cheyyi.

---

10. Helm template rendering is failing. What will you check?
First, I validate the Helm template syntax and YAML indentation.
Then I check missing values, incorrect template expressions, and invalid references.
I use Helm template rendering to see the generated Kubernetes YAML before applying it.
After fixing the template issue, I render it again and validate the output.
Simple ga gurthupettuko:
Template fail → syntax/YAML check → missing values check → rendered YAML chudu → fix and render again.
Eng: Template fails → check syntax and YAML → verify values → inspect rendered YAML → fix and render again.
Example:
helm lint ./mychart → Chart syntax and common chart issues unnaya check cheyyi.
helm template myapp ./mychart -f values.yaml → Template render chesthu exact error enti ani identify cheyyi.
cat values.yaml → Required values missing or wrong ga unnaya verify cheyyi.
grep -R "{{" ./mychart/templates → Template expressions and references ekkada use ayyayo inspect cheyyi.
Issue fix chesi → helm template myapp ./mychart -f values.yaml → Generated Kubernetes YAML correct ga render avutunda verify cheyyi.

---

11. Helm template renders successfully but Kubernetes deployment fails. What will you check?
First, I verify that the rendered YAML is valid for the Kubernetes cluster.
Then I check Kubernetes API errors, resource configuration, image, ports, probes, permissions, and required dependencies.
I also check Pod events after the resource is created.
Based on the Kubernetes error, I fix the manifest or values and upgrade again.
Simple ga gurthupettuko:
Template success but K8s fail → rendered YAML check → K8s error/events chudu → resource/image/config verify cheyyi → fix and upgrade.
Eng: Template succeeds but Kubernetes fails → check rendered YAML → inspect Kubernetes errors/events → verify resources and configuration → fix and upgrade.
Example:
helm template myapp ./mychart -f values.yaml → Kubernetes ki generate ayye final YAML correct ga undho check cheyyi.
kubectl get events -n dev --sort-by=.lastTimestamp → Deployment fail avvadaniki Kubernetes reason enti ani check cheyyi.
kubectl describe pod <pod_name> -n dev → Pod enduku start avvatledo events and configuration chudu.
kubectl logs <pod_name> -n dev → Application/container error logs check cheyyi.
Issue fix chesi → helm upgrade myapp ./mychart -n dev -f values.yaml → Deployment successful ga ayyindo verify cheyyi.

---

12. Wrong Docker image tag is deployed through Helm. What will you check?
First, I check the image repository and tag in the Helm values.
Then I verify whether another values file or --set parameter is overriding the expected tag.
I also check the rendered Deployment and the image actually running in the Pods.
After correcting the tag, I perform the Helm upgrade and verify the running version.
Simple ga gurthupettuko:
Wrong image → values lo tag check → override check → rendered Deployment chudu → running Pod version verify cheyyi.
Eng: Wrong image → check the tag in values → verify overrides → inspect the rendered Deployment → verify the running image.
Example:
grep -n "tag:" values.yaml → Helm values lo expected Docker image tag undho check cheyyi.
helm template myapp ./mychart -f values.yaml | grep "image:" → Final rendered Deployment lo ye image tag generate avutundo check cheyyi.
kubectl get deployment myapp -n dev -o yaml | grep "image:" → Kubernetes Deployment actually ye image use chestundo verify cheyyi.
kubectl get pods -n dev -o wide → New Pods create ayyaya ani check cheyyi.
Correct tag set chesi → helm upgrade myapp ./mychart -n dev -f values.yaml → Expected image running lo undho verify cheyyi.

---

13. Helm upgrade is deploying the old Docker image. What will you check?
First, I verify the image tag passed to Helm during the upgrade.
Then I check whether the registry contains that exact image and whether the Deployment is using the expected tag.
I also verify image pull behavior and the actual image running inside the new Pods.
I prefer immutable version tags instead of relying only on latest.
Simple ga gurthupettuko:
Old image → Helm tag check → registry image verify → Deployment/Pod image check → unique tag use cheyyi.
Eng: Old image → check the Helm tag → verify the registry image → check the Deployment and Pod image → use unique tags.
Example:
helm get values myapp -n dev → Running Helm release lo image tag enti ani check cheyyi.
kubectl get deployment myapp -n dev -o jsonpath='{.spec.template.spec.containers[*].image}' → Deployment lo configured image tag verify cheyyi.
kubectl get pods -n dev -o jsonpath='{.items[*].spec.containers[*].image}' → Pods ki ye image configure ayyindo check cheyyi.

kubectl describe pod <pod_name> -n dev → Image pull details and events chudu.
Unique tag use chesi → helm upgrade myapp ./mychart -n dev --set image.tag=v2.0.1 → New image deploy ayyindo verify cheyyi.

---

14. Helm chart dependency is failing. What will you check?
First, I check the dependency definition and version in Chart.yaml.
Then I verify the configured Helm repository and whether the required chart version exists.
I also check dependency download and repository connectivity.
After fixing the version or repository issue, I update the dependencies and retry the deployment.
Simple ga gurthupettuko:
Dependency fail → Chart.yaml check → version/repository verify → dependency download check → fix and update.
Eng: Dependency fails → check Chart.yaml → verify version and repository → check dependency download → fix and update.
Example:
cat Chart.yaml → Required dependency name and version correct ga unnaya check cheyyi.
helm repo list → Required Helm repository configured ga undho check cheyyi.
helm repo update → Repository latest chart information refresh cheyyi.
helm dependency update ./mychart → Chart dependencies download/update avutunnaya verify cheyyi.
Issue fix ayyaka → helm upgrade myapp ./mychart -n dev → Dependency tho deployment successful ga ayyindo check cheyyi.

---

15. Helm repository is unavailable. What will you check?
First, I verify whether the repository URL is correct and reachable.
Then I check network connectivity, DNS, proxy settings, and repository authentication if required.
I also verify whether the repository itself is experiencing an outage.
After restoring repository access, I retry the Helm operation.
Simple ga gurthupettuko:
Repo unavailable → URL check → network/DNS/proxy check → authentication/repo health check → retry.
Eng: Repository unavailable → check the URL → verify network/DNS/proxy → check authentication and repository health → retry.
Example:
helm repo list → Configured repository URL correct ga undho check cheyyi.
helm repo update → Repository reachable ga undho and metadata download avutunda check cheyyi.
nslookup <repo-domain> → Repository domain DNS resolve avutunda verify cheyyi.
curl -I <repo-url> → Repository URL network nunchi reachable ga undho check cheyyi.
Issue fix chesi → helm repo update → Repository access restore ayyindo verify cheyyi.

---

16. Helm release shows deployed, but the application is not accessible. What will you check?
First, I verify the Helm release resources and check whether the Pods are actually healthy.
Then I check the Service, ports, selectors, Endpoints, and Ingress configuration.
I also verify application logs and readiness probes.
Based on where traffic is failing, I fix the Kubernetes resource configuration.
Simple ga gurthupettuko:
Release deployed but access ledu → Pods check → Service/ports/Endpoints check → Ingress/probe verify → traffic issue fix.
Eng: Release deployed but inaccessible → check Pods → verify Service, ports, and Endpoints → check Ingress and probes → fix the traffic issue.
Example:
helm status myapp -n prod → Helm release successfully deployed status lo undho check cheyyi.
kubectl get pods -n prod → Application Pods Ready ga unnaya verify cheyyi.
kubectl get svc myapp -n prod → Service port and target port correct ga unnaya check cheyyi.
kubectl get endpoints myapp -n prod → Service ki healthy Pod endpoints attach ayyaya verify cheyyi.
kubectl get ingress -n prod → Ingress routing configuration check cheyyi.
Issue fix chesi → kubectl rollout status deployment/myapp -n prod → Application rollout healthy ga complete ayyindo verify cheyyi.

---

17. Helm upgrade causes Pods to restart unexpectedly. What will you check?
First, I compare the old and new rendered Kubernetes manifests.
Then I check whether image, configuration, environment variables, probes, or resource values changed.
I also check the Deployment rollout and Pod events.
If the change is causing instability, I fix the configuration or roll back the release.
Simple ga gurthupettuko:
Upgrade tarvatha restart → old/new manifests compare → image/config/probe/resource changes check → fix or rollback.
Eng: Pods restart after upgrade → compare old and new manifests → check image, config, probes, and resources → fix or roll back.
Example:
helm get manifest myapp -n prod → Current release generate chesina Kubernetes manifest chudu.
helm history myapp -n prod → Previous working revision identify cheyyi.
kubectl rollout history deployment/myapp -n prod → Deployment rollout changes check cheyyi.
kubectl get events -n prod --sort-by=.lastTimestamp → Pod restarts ki Kubernetes reason enti ani check cheyyi.
Problematic change unte → helm rollback myapp <revision> -n prod → Previous stable release ki rollback cheyyi.

---

18. Helm deployment is partially successful. What will you check?
First, I identify which Kubernetes resources were successfully created or updated.
Then I check the failed resources, Pod events, Helm status, and release history.
I also verify whether the failure is related to configuration, image, resources, or dependencies.
Based on the state, I either complete the upgrade or safely roll back.

Simple ga gurthupettuko:
Partial success → successful/failed resources identify cheyyi → events/status chudu → failure reason find cheyyi → complete or rollback.
Eng: Partial deployment → identify successful and failed resources → check events and status → find the failure → complete or roll back.
Example:
helm status myapp -n dev → Release current state and resources status check cheyyi.
kubectl get all -n dev → Which resources create/update ayyayo identify cheyyi.
kubectl get events -n dev --sort-by=.lastTimestamp → Failed resource ki exact Kubernetes error chudu.
kubectl describe pod <pod_name> -n dev → Failed Pod configuration and events inspect cheyyi.
Issue fix ayithe → helm upgrade myapp ./mychart -n dev → Remaining deployment complete ayyindo verify cheyyi.

---

19. Helm hook is causing deployment failure. What will you check?
First, I identify which hook is failing and check its logs and Kubernetes events.
Then I verify the hook's image, permissions, configuration, and dependencies.
I also check whether the hook is blocking the Helm operation.
After fixing the hook issue, I rerun the Helm operation and verify the release.
Simple ga gurthupettuko:
Hook fail → hook identify → logs/events chudu → image/permissions/config check → fix and retry.
Eng: Hook failure → identify the hook → check logs and events → verify image, permissions, and config → fix and retry.
Example:
helm status myapp -n dev → Release hook valla operation fail ayyinda check cheyyi.
kubectl get jobs -n dev → Hook Job create ayyinda and status enti ani check cheyyi.
kubectl logs job/<job_name> -n dev → Hook Job exact error logs chudu.
kubectl describe job <job_name> -n dev → Job events and failure reason verify cheyyi.
Issue fix chesi → helm upgrade myapp ./mychart -n dev → Hook successfully complete ayyi release deploy ayyindo verify cheyyi.

---

20. Two Helm releases are conflicting with the same Kubernetes resource. What will you check?
First, I identify which resource both releases are trying to manage.
Then I check the release names, namespaces, resource ownership, and chart configuration.
I verify whether the same resource is hardcoded in both charts.
I separate the resource ownership or correct the chart configuration to avoid conflicts.
Simple ga gurthupettuko:
Resource conflict → common resource identify → releases/namespaces check → ownership check → chart configuration fix.
Eng: Resource conflict → identify the common resource → check releases and namespaces → verify ownership → fix the chart configuration.
Example:
helm list -A → Cluster lo unna Helm releases anni identify cheyyi.
kubectl get deployment -A → Same resource name different releases lo conflict avutunda check cheyyi.
helm get manifest myapp -n dev → First release ye resource manage chestundo inspect cheyyi.
helm get manifest myapp2 -n dev → Second release kuda same resource create chestundo compare cheyyi.
Conflict fix chesi → helm upgrade myapp ./mychart -n dev → Resource ownership conflict lekunda deployment verify cheyyi.

---

21. Helm release was accidentally deleted. What will you do?
First, I assess the impact and check whether the Kubernetes resources are still running.
Then I check Helm history or backups to identify the previous release configuration.
If required, I restore the release using the correct chart version and values.
After recovery, I verify the application and investigate how the deletion happened.
Simple ga gurthupettuko:
Release delete ayithe → impact check → old chart/values identify → restore/reinstall → app verify → root cause check.
Eng: Release deleted → assess impact → identify the previous chart and values → restore the release → verify the application.
Example:
helm list -A → Release actually delete ayyinda ani verify cheyyi.
kubectl get pods -n prod → Kubernetes resources still running unnaya check cheyyi.
helm history myapp -n prod → Available release history tho previous revision information check cheyyi.
helm install myapp ./mychart -n prod -f values-prod.yaml → Required ayithe correct chart and values tho release restore cheyyi.
kubectl get pods -n prod → Recovery tarvatha application Pods healthy ga unnaya verify cheyyi.

---

22. How do you safely upgrade a Production Helm release?
First, I validate the chart and values in a lower environment before Production.
Then I verify the rendered manifests, image version, configuration, and rollback revision.
During Production, I perform a controlled upgrade and monitor the rollout.
If the release causes issues, I use the known-good revision for rollback.
Simple ga gurthupettuko:
Prod upgrade → lower env test → manifests/image/config verify → controlled upgrade → monitor → rollback if needed.
Eng: Production upgrade → test first → verify manifests and image → perform a controlled upgrade → monitor → roll back if needed.
Example:
helm upgrade myapp ./mychart -n qa -f values-qa.yaml → First lower environment lo same chart and values test cheyyi.
helm template myapp ./mychart -f values-prod.yaml → PROD ki generate ayye final manifests verify cheyyi.
helm history myapp -n prod → Rollback kosam previous good revision note chesi petko.
helm upgrade myapp ./mychart -n prod -f values-prod.yaml → Controlled PROD upgrade perform cheyyi.

kubectl rollout status deployment/myapp -n prod → Production rollout successful ga complete ayyindo monitor cheyyi.
Issue vaste → helm rollback myapp <revision> -n prod → Known-good revision ki rollback cheyyi.

---

23. How do you verify a Helm deployment before Production rollout?
First, I validate the chart templates and render the final Kubernetes manifests.
Then I verify image versions, configuration, resources, probes, Services, and environment-specific values.
I also test the same chart and values in a lower environment.
Only after validation do I promote the release to Production.
Simple ga gurthupettuko:
Prod mundu → template/render check → image/config/resources verify → lower env test → then Prod deploy.
Eng: Before Production → validate templates → verify image/config/resources → test in a lower environment → deploy to Production.
Example:
helm lint ./mychart → Chart syntax and configuration issues unnaya check cheyyi.
helm template myapp ./mychart -f values-prod.yaml → PROD final YAML correct ga generate avutunda verify cheyyi.
helm upgrade myapp ./mychart -n qa -f values-qa.yaml → Lower environment lo deployment test cheyyi.
kubectl rollout status deployment/myapp -n qa → QA rollout successful ga complete ayyindo verify cheyyi.
Validation successful ayyaka → helm upgrade myapp ./mychart -n prod -f values-prod.yaml → PROD deployment perform cheyyi.

---

24. How do you troubleshoot Helm + Kubernetes deployment failure?
First, I check the Helm command output and release status to identify where the failure started.
Then I inspect the rendered manifests, Kubernetes events, Pod logs, and resource status.
I trace the issue from Helm values to the generated manifest and finally to the running Pod.
After identifying the root cause, I fix the values or chart and perform a controlled upgrade.
Simple ga gurthupettuko:
Helm + K8s fail → Helm error chudu → rendered YAML check → K8s events/logs chudu → values/chart fix → upgrade.
Eng: Helm + Kubernetes failure → check Helm error → inspect rendered YAML → check Kubernetes events/logs → fix the chart or values → upgrade.
Example:
helm status myapp -n dev → Helm release status and failure point check cheyyi.
helm template myapp ./mychart -f values.yaml → Generated Kubernetes YAML inspect cheyyi.
kubectl get events -n dev --sort-by=.lastTimestamp → Kubernetes deployment failure reason chudu.
kubectl logs <pod_name> -n dev → Application/container error identify cheyyi.
Issue fix chesi → helm upgrade myapp ./mychart -n dev -f values.yaml → Controlled upgrade run chesi verify cheyyi.

---

25. How do you manage Helm chart versions in Production?
I maintain chart versions separately from application versions so changes can be tracked clearly.
Before upgrading, I identify the current Production chart version and the target version.
I test the new chart version in a lower environment and keep the previous version available for rollback.
This gives us controlled upgrades and a clear recovery path.
Simple ga gurthupettuko:
Chart version → current version note cheyyi → new version test cheyyi → Prod upgrade cheyyi → old version rollback kosam ready ga unchuko.
Eng: Chart version → record the current version → test the new version → upgrade Production → keep the old version for rollback.
Example:
helm list -n prod → Current PROD release and chart version check cheyyi.
helm show chart ./mychart → New chart version enti ani verify cheyyi.
helm upgrade myapp ./mychart -n qa -f values-qa.yaml → New chart version ni QA lo test cheyyi.
helm history myapp -n prod → Previous PROD revisions rollback kosam identify cheyyi.
Validation ayyaka → helm upgrade myapp ./mychart -n prod -f values-prod.yaml → Approved chart version PROD ki deploy cheyyi.

---

26. Helm deployment is using unexpected replicas or resources. What will you check?
First, I check the values used for replicas, CPU, and memory.
Then I verify whether another values file or --set option is overriding those values.
I also check the rendered Deployment to confirm what Kubernetes actually receives.
After finding the incorrect value, I correct it and upgrade the release.
Simple ga gurthupettuko:
Wrong replicas/resources → values check → override check → rendered Deployment verify → correct and upgrade.
Eng: Wrong replicas/resources → check values → verify overrides → inspect the rendered Deployment → correct and upgrade.
Example:
grep -n "replicaCount" values.yaml → Expected replica count values file lo undho check cheyyi.
grep -n "resources:" values.yaml → CPU and memory configuration correct ga undho verify cheyyi.
helm get values myapp -n dev → Running release lo actual values enti ani check cheyyi.
helm template myapp ./mychart -f values.yaml | grep -E "replicas|cpu|memory" → Rendered manifest lo Kubernetes ki actual values enti ani verify cheyyi.
Correct values set chesi → helm upgrade myapp ./mychart -n dev -f values.yaml → Replicas and resources expected ga apply ayyayo verify cheyyi.

---

27. Helm chart works on one Kubernetes cluster but fails on another. What will you check?
First, I compare the Kubernetes versions and cluster configurations.
Then I check API versions, CRDs, permissions, admission policies, namespaces, and available resources.

I also compare the Helm chart version and values used in both clusters.
Based on the difference, I update the chart or cluster configuration.
Simple ga gurthupettuko:
One cluster work, another fail → K8s versions compare → API/CRD/RBAC check → values/chart compare → difference fix.
Eng: Works on one cluster but fails on another → compare Kubernetes versions → check APIs, CRDs, and RBAC → compare chart and values → fix the difference.
Example:
kubectl version → Both clusters Kubernetes versions compare cheyyi.
kubectl api-resources → Required API resources cluster lo available unnaya check cheyyi.
kubectl get crd → Required CRDs installed unnaya verify cheyyi.
kubectl auth can-i create deployments -n prod → Current user ki required permissions unnaya check cheyyi.
helm version → Helm versions compare cheyyi.
Difference identify chesi → required chart/configuration fix chesi → helm upgrade myapp ./mychart -n prod → Second cluster deployment verify cheyyi.

---

28. Helm Secret/config changes are not reflected after upgrade. What will you check?
First, I verify that the new Secret or ConfigMap values are actually present in the rendered manifest.
Then I check whether the application consumes them through environment variables or mounted files.
If environment variables are used, the Pods may need to restart to receive the new values.
I safely restart or roll out the workload and verify the updated configuration.
Simple ga gurthupettuko:
Config change reflect avvakapothe → rendered values check → Pod ela consume chestundo chudu → required ayithe restart → verify.
Eng: Config change not reflected → verify rendered values → check how the Pod consumes them → restart if required → verify the update.
Example:
helm template myapp ./mychart -f values.yaml | grep -A5 "ConfigMap" → New configuration rendered manifest lo undho check cheyyi.
kubectl get configmap <configmap_name> -n dev -o yaml → Kubernetes ConfigMap lo updated value undho verify cheyyi.
kubectl get secret <secret_name> -n dev → Required Secret present ga undho check cheyyi.
kubectl rollout restart deployment/myapp -n dev → Environment variables refresh avvalante Pods ni safely restart cheyyi.
kubectl rollout status deployment/myapp -n dev → Restart/rollout successful ga complete ayyindo verify cheyyi.

---

29. How do you prevent accidental Production Helm deployments?
I separate Production values and credentials from lower environments.
I also restrict Production access and use approval controls in the CI/CD pipeline.
Production releases should use controlled chart versions and approved branches or tags.
This reduces the chance of an accidental Helm deployment.
Simple ga gurthupettuko:
Accidental Prod deploy avoid → Prod access restrict → credentials separate → approval add → approved version matrame deploy cheyyi.
Eng: Prevent accidental Production deployment → restrict access → separate credentials → add approval → deploy only approved versions.
Example:
kubectl auth can-i create deployments -n prod → Current user ki PROD deployment permission undho check cheyyi.
helm list -n prod → PROD lo existing releases and versions verify cheyyi.
helm upgrade myapp ./mychart -n prod -f values-prod.yaml --dry-run → Actual deployment mundu proposed changes inspect cheyyi.
CI/CD pipeline lo approval stage → Production deployment ki authorized approval required ga enforce cheyyi.
Approval tarvatha → helm upgrade myapp ./mychart -n prod -f values-prod.yaml → Approved version matrame deploy cheyyi.

---

30. Real-time CI/CD → Helm → Kubernetes deployment scenario. Explain.
Developer code ni Git ki push chestadu and CI pipeline build/test complete chestundi.
Docker image ni versioned tag tho build chesi registry ki push chestham.
Helm values lo new image tag update chesi Helm chart ni Kubernetes cluster ki deploy chestham.
After deployment, Pods, rollout status, health checks, and monitoring verify chestham; issue vaste previous Helm revision ki rollback chestham.
gurthupettuko:Git → CI → Docker image → Registry → Helm values/chart → Kubernetes → Pods → Health check → Rollback.
Eng: Git → CI → Docker image → Registry → Helm chart → Kubernetes → Pods → health check → rollback.
Example:
git push origin main → Developer latest code ni Git repository ki push chestadu, CI pipeline trigger avutundi.
docker build -t myapp:v1.2.0 . → Application code tho versioned Docker image build chestham.
docker push registry.example.com/myapp:v1.2.0 → Built image ni container registry ki push chestham.
helm upgrade --install myapp ./mychart -n prod --set image.tag=v1.2.0 → New image tag tho Helm chart ni Kubernetes PROD cluster ki deploy chestham.
kubectl rollout status deployment/myapp -n prod → Deployment rollout successful ga complete ayyindo verify chestham.
kubectl get pods -n prod → Pods Running/Ready lo unnaya check chestham.
Issue vaste → helm rollback myapp <revision> -n prod → Previous known-good Helm revision ki rollback chestham.